

Data Structures – Practice Questions with Answers

Q1. What is a Data Structure? Give 3 real-life examples.

A Data Structure is a way of organizing, storing, and managing data in a computer so it can be accessed and modified efficiently. It defines the relationship between data elements and the operations that can be performed on them.

Real-life examples:

1. A stack of plates in a kitchen (Stack – Last In First Out).
2. A queue of people waiting at a ticket counter (Queue – First In First Out).
3. A phone's contact list organized for quick search (Array / Hash Table).

Q2. Differentiate between Linear and Non-linear Data Structures.

Linear Data Structures: Elements are arranged in a sequential order, where each element is connected to its previous and next element. Examples: Array, Linked List, Stack, Queue. They are easy to implement, but memory utilization is less efficient for non-sequential relationships, and traversal is single-level (one pass).

Non-linear Data Structures: Elements are not arranged sequentially; each element can be connected to multiple other elements, forming a hierarchical or networked structure. Examples: Tree, Graph. They represent complex relationships efficiently and traversal may require multiple paths (e.g., BFS, DFS), but they are generally more complex to implement.

Q3. Explain the internal structure of an Array with a memory diagram.

An array is a collection of elements of the same data type stored in contiguous memory locations. Each element can be accessed directly using its index, and the address of any element can be calculated using the formula: $\text{Address}(i) = \text{Base Address} + (i \times \text{Size of each element})$.

Memory Diagram (Array: $A[5] = \{10, 20, 30, 40, 50\}$, Base Address = 1000, each int = 4 bytes):

Index:	0	1	2	3	4
Value:	10	20	30	40	50
Address:	1000	1004	1008	1012	1016

Since memory locations are contiguous, accessing any element by index takes constant time, $O(1)$.

Q4. Write the algorithm and C program for Array insertion.

Algorithm (Insert element 'x' at position 'pos' in array of size 'n'):

1. Start.
2. Check if the array is full; if full, terminate (overflow).
3. Shift all elements from the last index up to 'pos' one position to the right.
4. Place 'x' at index 'pos'.
5. Increase the count of elements (n) by 1.

6. Stop.

C Program:

```
#include <stdio.h>
int main() {
    int arr[10] = {10, 20, 30, 40, 50};
    int n = 5, pos = 2, x = 99, i;
    for (i = n; i > pos; i--) {
        arr[i] = arr[i - 1];
    }
    arr[pos] = x;
    n++;
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

Q5. Explain Stack using LIFO concept with a diagram.

A Stack is a linear data structure that follows the LIFO (Last In, First Out) principle, meaning the element inserted last is the first one to be removed. Insertion and deletion both happen at one end, called the 'top' of the stack. The main operations are Push (insert) and Pop (remove).

Diagram (pushing 10, 20, 30 in order):

```
| 30 | ← Top (last in, first out)
| 20 |
| 10 | ← Bottom
```

When a Pop operation is performed, 30 (the last element pushed) is removed first.

Q6. What is the difference between Stack and Queue?

Stack: Follows LIFO (Last In, First Out) order. Insertion and deletion happen at the same end (the 'top'). Main operations are Push and Pop. Example use: function call management (call stack), undo operations in editors.

Queue: Follows FIFO (First In, First Out) order. Insertion happens at the 'rear' and deletion happens at the 'front' — two different ends. Main operations are Enqueue and Dequeue. Example use: task scheduling, printer queues, handling requests in order of arrival.

Q7. Write C code for Enqueue and Dequeue operations.

```
#define SIZE 5
int queue[SIZE], front = -1, rear = -1;

void enqueue(int x) {
    if (rear == SIZE - 1) {
        printf("Queue Overflow");
    }
}
```

```

        return;
    }
    if (front == -1) front = 0;
    queue[++rear] = x;
}

int dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow");
        return -1;
    }
    return queue[front++];
}

```

Enqueue adds an element at the rear of the queue (after checking for overflow), while Dequeue removes an element from the front of the queue (after checking for underflow).

Q8. Explain Singly Linked List with a node diagram.

A Singly Linked List is a linear data structure made up of nodes, where each node contains two parts: a data field that stores the value, and a next pointer that stores the address of the next node in the sequence. The last node's next pointer points to NULL, indicating the end of the list. Unlike arrays, linked list elements are not stored in contiguous memory, and traversal can only happen in one direction (forward).

Node Structure:

| Data | Next | →

Diagram (List: 10 → 20 → 30 → NULL):

[Head] → |10|*| → |20|*| → |30|/| → NULL

Here, Head points to the first node. Each node's pointer links to the next node, and the final node points to NULL.